# Intermediate Report on working with project "House Prices — Advanced Regression Techniques" (Kaggle competition)

**Author: Vsevolod Muronets**

The chosen project is Kaggle's competition "House Prices — Advanced Regression Techniques". The goal of the project is to build a model that will most precisely predict the sale prices of houses in the test dataset using machine learning techniques. The suggested machine learning model to use is TensorFlow Decision Trees, but since it was not covered in our course, I use what we have already discussed — Linear Regression model from Scikit-learn (Sklearn) library with "SalePrice" as the dependent variable and other variables as independent. It may provide less correct predictions than TensorFlow Decision Trees (although that is yet to be tested, however), but will constitute a better learning practice.

To begin with the project, I load the training dataset as a Pandas DataFrame and look at its features. The training data contains 1460 observations (rows) and 81 features. As can be seen from the table below (Table 1), 38 of them are (at least seemingly) numerical, while other 43 are categorical (that is, of "object" datatype). The column 'Id' contains ID's of observations and is of no use for regression modeling. Thus, this column should be excluded from further analysis, so I drop it from the training dataset that I will work with. 37 numerical variables remain.

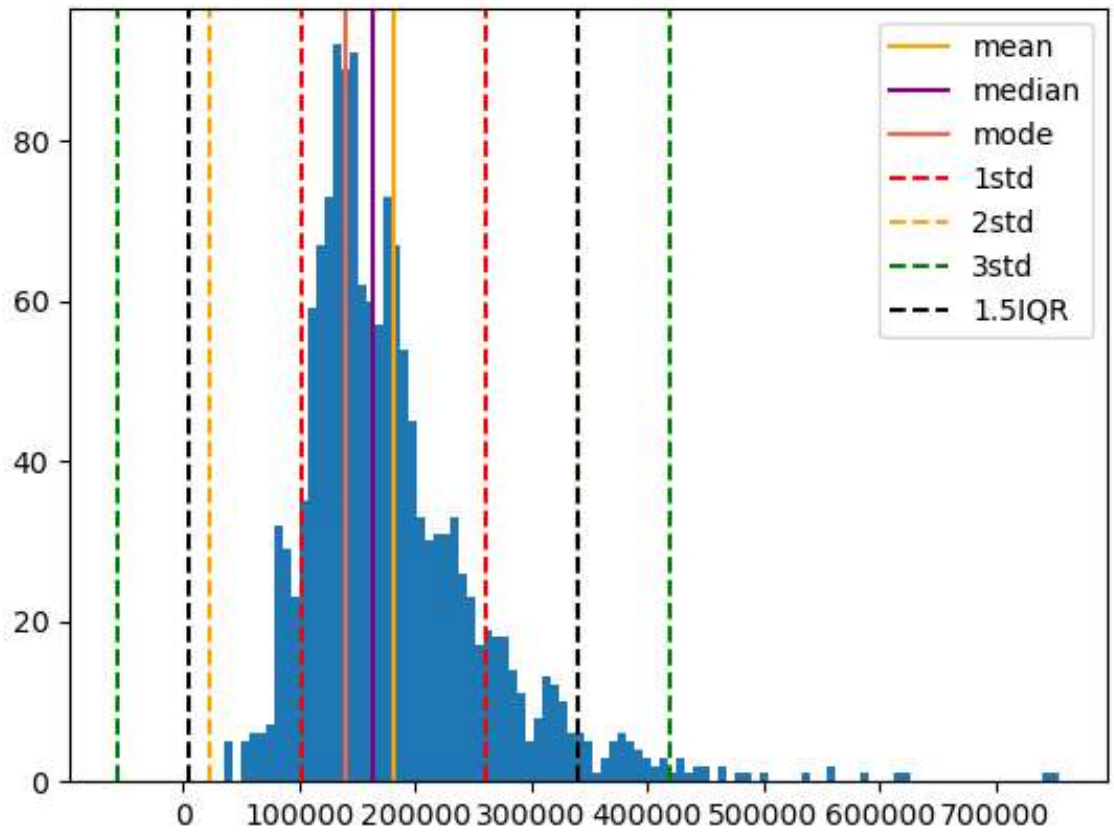
| Variable name | Value type | Variable name | Value type |
|---|---|---|---|
| **Id** | int64 | **CentralAir** | object |
| **MSSubClass** | int64 | **Electrical** | object |
| **MSZoning** | object | **1stFlrSF** | int64 |
| **LotFrontage** | float64 | **2ndFlrSF** | int64 |
| **LotArea** | int64 | **LowQualFinSF** | int64 |
| **Street** | object | **GrLivArea** | int64 |
| **Alley** | object | **BsmtFullBath** | int64 |
| **LotShape** | object | **BsmtHalfBath** | int64 |
| **LandContour** | object | **FullBath** | int64 |

| | | | |
|---|---|---|---|
| **Utilities** | object | **HalfBath** | int64 |
| **LotConfig** | object | **BedroomAbvGr** | int64 |
| **LandSlope** | object | **KitchenAbvGr** | int64 |
| **Neighborhood** | object | **KitchenQual** | object |
| **Condition1** | object | **TotRmsAbvGrd** | int64 |
| **Condition2** | object | **Functional** | object |
| **BldgType** | object | **Fireplaces** | int64 |
| **HouseStyle** | object | **FireplaceQu** | object |
| **OverallQual** | int64 | **GarageType** | object |
| **OverallCond** | int64 | **GarageYrBlt** | float64 |
| **YearBuilt** | int64 | **GarageFinish** | object |
| **YearRemodAdd** | int64 | **GarageCars** | int64 |
| **RoofStyle** | object | **GarageArea** | int64 |
| **RoofMatl** | object | **GarageQual** | object |
| **Exterior1st** | object | **GarageCond** | object |
| **Exterior2nd** | object | **PavedDrive** | object |
| **MasVnrType** | object | **WoodDeckSF** | int64 |
| **MasVnrArea** | float64 | **OpenPorchSF** | int64 |
| **ExterQual** | object | **EnclosedPorch** | int64 |
| **ExterCond** | object | **3SsnPorch** | int64 |
| **Foundation** | object | **ScreenPorch** | int64 |
| **BsmtQual** | object | **PoolArea** | int64 |
| **BsmtCond** | object | **PoolQC** | object |
| **BsmtExposure** | object | **Fence** | object |
| **BsmtFinType1** | object | **MiscFeature** | object |
| **BsmtFinSF1** | int64 | **MiscVal** | int64 |
| **BsmtFinType2** | object | **MoSold** | int64 |
| **BsmtFinSF2** | int64 | **YrSold** | int64 |
| **BsmtUnfSF** | int64 | **SaleType** | object |
| **TotalBsmtSF** | int64 | **SaleCondition** | object |
| **Heating** | object | **SalePrice** | int64 |
| **HeatingQC** | object | | |

**Table 1.** *Variables in the dataset and corresponding variable types.*

The dependent variable that we will work with is “SalePrice”, it is of integer type and is measured in dollars. As it can be seen from the histogram below (Figure

1), the distribution seems to be positively skewed, although it resembles a normal one if we are not taking into account the outliers. There are quite many outliers (even above the upper three standard deviations threshold; there are 22 of them, and 0 below the 3std-threshold), which are highly expensive houses, and since it will probably be required to predict prices of such expensive houses also and there are many of them, I will leave them in the dataset that the model will be trained on.




**Picture 1.** *Distribution of the dependent variable ("SalePrice").*

The next essential step is to deal with missing values in the training dataset, since the basic linear regression model from the Sklearn library that we have covered on the seminars does not seem to be able to process them naturally. As can be seen from the code notebook and the table below (Table 2), for 19 variables missing values are present, and 5 of them even contain missing values more than half of the whole

amount of observations (that is, more than 730 missing values). Processing them requires us to handle each variable individually. Thus, “PoolQC” missing variables simply mean a lack of pool in the house, as it is explained in dataset description on its Kaggle page. Missing values should be recoded correspondingly (I recode it as “NoPool”). The situation is similar for other categorical variables such as “MiscFeature” (lack of miscellaneous features in the house), “Alley” (lack of alley access), “Fence” (lack of fences), “MasVnrType” (lack of masonry veneer layer on walls), “FireplaceQu” (lack of fireplace), and various categorical variables connected to basement (“BsmtExposure”, “BsmtFinType2”, “BsmtFinType2”, “BsmtQual”, “BsmtCond”, and “BsmtFinType1”) and to garage (“GarageFinish”, “GarageCond”, “GarageType”, and “GarageQual”). Equal numbers of missing values for the last two groups of variables (81 for garage-related and 37-38 for basement-related) proves that missing values appeared there intentionally. For the “Electrical” feature, missing value also probably means that the house does not have electricity, but I do not think that recoding it is necessary since there is only one observation with missing value for this feature, and deleting the observation will not do the dataset any harm (it would be generally interesting how lack of electricity may influence sale price of the house, but one observation will, of course, not allow us to draw any statistically significant conclusions anyway). For a numerical garage-related variable “GarageYrBlt” (year that the garage was built) missing values also most probably signify a lack of garage in the house, and since we cannot equate it to 0 (it would spoil the calculations since the garages that were indeed build were built in the 20-21st centuries, as I have checked using *.unique()* function, although the cell was deleted from the notebook, because I considered it to be unnecessary), I equated it to the median value for the column. “MasVnrArea” and “LotFrontage” I equated to zero, since I think that missing values there also signify lack of masonry veneer and no access to the street for the house correspondingly, and so the area for both variables is 0.

| Variable name | Number of missing values | Variable name | Number of missing values |
|---|---|---|---|
| PoolQC | 1453 | LotShape | 0 |
| MiscFeature | 1406 | LandContour | 0 |
| Alley | 1369 | Utilities | 0 |
| Fence | 1179 | LotConfig | 0 |
| MasVnrType | 872 | LandSlope | 0 |
| FireplaceQu | 690 | Neighborhood | 0 |
| LotFrontage | 259 | Condition1 | 0 |
| GarageFinish | 81 | BldgType | 0 |
| GarageCond | 81 | MSZoning | 0 |
| GarageType | 81 | HouseStyle | 0 |
| GarageYrBlt | 81 | OverallQual | 0 |
| GarageQual | 81 | OverallCond | 0 |
| BsmtExposure | 38 | YearBuilt | 0 |
| BsmtFinType2 | 38 | YearRemodAdd | 0 |
| BsmtQual | 37 | RoofStyle | 0 |
| BsmtCond | 37 | RoofMatl | 0 |
| BsmtFinType1 | 37 | Exterior1st | 0 |
| MasVnrArea | 8 | Exterior2nd | 0 |
| Electrical | 1 | HalfBath | 0 |
| TotalBsmtSF | 0 | PoolArea | 0 |
| BsmtFinSF1 | 0 | BedroomAbvGr | 0 |
| BsmtFinSF2 | 0 | MiscVal | 0 |
| BsmtUnfSF | 0 | 3SsnPorch | 0 |
| HeatingQC | 0 | MoSold | 0 |
| Heating | 0 | YrSold | 0 |
| ExterCond | 0 | SaleType | 0 |
| CentralAir | 0 | SaleCondition | 0 |
| 1stFlrSF | 0 | ScreenPorch | 0 |
| 2ndFlrSF | 0 | WoodDeckSF | 0 |
| LowQualFinSF | 0 | EnclosedPorch | 0 |
| GrLivArea | 0 | OpenPorchSF | 0 |
| BsmtFullBath | 0 | KitchenAbvGr | 0 |
| BsmtHalfBath | 0 | PavedDrive | 0 |
| FullBath | 0 | GarageArea | 0 |

| | | | |
|---|---|---|---|
| **Foundation** | 0 | **GarageCars** | 0 |
| **MSSubClass** | 0 | **Fireplaces** | 0 |
| **ExterQual** | 0 | **Functional** | 0 |
| **Condition2** | 0 | **TotRmsAbvGrd** | 0 |
| **LotArea** | 0 | **KitchenQual** | 0 |
| **Street** | 0 | **SalePrice** | 0 |

**Table 2.** *Variables from the dataset and number of corresponding missing values for each variable.*

When the missing values are dealt with, I split the general dataset in two, into training and testing ones. For the testing dataset, I leave approximately 30% of all of the observations, and approximately 60% for the training dataset. The competition also contains a separate dataset for testing, but it does not contain the values that I need to predict (values for "SalePrice"), so for preliminary testing and evaluation of the model it is crucial to split the original training dataset and only then train the model on the full training dataset when the model works well with the split one.

As can be seen from the notebook, the mean squared error (MSE) is quite big (405955872.60639364), meaning that the model is probably overfitted. Fixing the overfitting, evaluating the corrected model and gaining prediction constitutes the remaining half (although I would say that it is probably less than a half) of the general work on the project, so I am leaving out from the intermediate report on the progress.